STEM Fusion

Introduction to Computational Thinking and Python Programming

LOGIC AND CONTROL FLOW

Boolean Logic · If Statements · For Loops · While Loops · Break · Continue

Instructor: Sarom Leang, PhD
Specialist: Daniela Garcia Galindo
Innovation Hall 129

Pronouns

Instructor

Sarom Leang, Ph.D.

Pronouns: Teacher / Instructor

Specialist

Daniela Garcia Galindo

Pronouns: Daniela

Schedule

	10:00 AM - 10:15 AM	Homeroom
	10:15 AM - 11:35 AM	G1
	11:40 AM - 12:35 PM	Lunch
	12:40 PM - 02:00 PM	G2

Student Expectations

🚫 **NO FOOD**	🥤 **NO DRINKS (on the table)**	🤝 Be respectful to individuals and property
🌱 Be open to learning	🤔 Be open to not understanding	🧘‍♂️ Be patient with yourself
✋ Ask questions	🔍 Explore	💡 Embrace failure

Course Overview

This course introduces the fundamental building blocks of computational thinking and computer programming using the Python language.

Upon successful completion of this course, students will be able to:

- Improve their problem-solving skills
- Write, read, and execute Python code using basic data types and operators

Course Overview (cont.)

Exploratory Topics

- ~~Session #1 – November 1, 2025~~
  - ~~Entrance Survey~~
  - ~~How Computers Work~~
- ~~Session #2 – December 6, 2025~~
  - ~~Parallel Computing~~
- ~~Session #3 – January 31, 2025~~
  - ~~Generative Artificial Intelligence~~
- ~~Session #4 – March 7, 2026~~
  - ~~Generative Artificial Intelligence~~
- Session #5 – April 11, 2026
  - Future of Computing (Quantum)
  - Exit survey

Topics

- Algorithms and Problem Solving
- Debugging and Troubleshooting
- Loops
- Algorithms and Syntax
- Variables and Conditionals
- Variable Arithmetic
- Conditionals (If/Else)
- Compound Conditionals

What Are We Doing Today? 

1	2	3
✅	🔀	🔁
Boolean Logic	If Statements	For Loops

4	5	6
⌛	🛑	⏭️
While Loops	Break	Continue

Boolean Logic

The universe of True and False — every decision starts here

What is a Boolean? True or False 🎯

Song in playlist?

`True`

Game over?

`False`

User signed in?

`True`

Price under $50?

`False`

Answer correct?

`True`

```
In Python: True and False are capitalized. type(True) is <class 'bool'>.
```

Comparison Operators

Comparison operators evaluate a condition and always return True or False.

`==` Equal to

```
5 == 5  →  True
```

`!=` Not equal to

```
5 != 3  →  True
```

`>` Greater than

```
7 > 3  →  True
```

`<` Less than

```
3 < 7  →  True
```

`>=` Greater than or equal

```
5 >= 5  →  True
```

`<=` Less than or equal

```
4 <= 4  →  True
```

Logical Operators: and, or, not 🧩

and

True only if BOTH sides are True

```
age >= 13 and has_account
# True only if both are True
```

True and True	**True**
True and False	**False**
False and False	**False**

or

True if AT LEAST ONE side is True

```
is_admin or is_premium
# True if either is True
```

True or True	**True**
True or False	**True**
False or False	**False**

not

Flips True to False and vice versa

```
not is_banned
# True if is_banned is False
```

not True	**False**
not False	**True**

Logical Operators in Action 🔎

Combining conditions with and / or / not — predict True or False before reading the answer!

and — concert access — True

```
age = 16
has_ticket = True

age >= 13 and has_ticket
# True — both are True
```

and — requires both — False

```
score = 85
attended = False

score >= 60 and attended
# False — attended is False
```

or — free entry — True

```
is_member = False
has_coupon = True

is_member or has_coupon
# True — at least one True
```

or — neither works — False

```
is_vip = False
is_staff = False

is_vip or is_staff
# False — both are False
```

not — flip it — True

```
is_banned = False


not is_banned
# True — flips False → True
```

compound — range check — True

```
temp = 72


temp >= 60 and temp <= 80
# True — 72 is in range
```

⚡ OPTIONAL CHALLENGE — Boolean Logic

✏️ Pen & Paper

Without running the code, predict True or False for each:

1. 10 > 5
2. 7 == 7.0
3. 'cat' != 'dog'
4. not True
5. (3 > 1) and (2 < 1)
6. (5 == 5) or (10 < 2)
7. not (4 >= 4)

Bonus: write a boolean expression that checks whether a score (set to any number you choose) is passing (>= 60) AND below a perfect score (< 100).

💻 Try it in IDLE

```
# Test your predictions in IDLE!
print(10 > 5)
print(7 == 7.0)
print('cat' != 'dog')
print(not True)
print((3 > 1) and (2 < 1))
print((5 == 5) or (10 < 2))
print(not (4 >= 4))

# Bonus:
score = 85
print(score >= 60 and score < 100)
```

Work individually • There's no wrong answer — just think it through!

✅ ANSWERS — Boolean Logic

✏️ Pen & Paper

Without running the code, predict True or False for each:

1. 10 > 5
2. 7 == 7.0
3. 'cat' != 'dog'
4. not True
5. (3 > 1) and (2 < 1)
6. (5 == 5) or (10 < 2)
7. not (4 >= 4)

Bonus: write a boolean expression that checks whether a score (set to any number you choose) is passing (>= 60) AND below a perfect score (< 100).

💻 IDLE Output

```
>>> print(10 > 5)
True
>>> print(7 == 7.0)
True
>>> print('cat' != 'dog')
True
>>> print(not True)
False
>>> print((3 > 1) and (2 < 1))
False
>>> print((5 == 5) or (10 < 2))
True
>>> print(not (4 >= 4))
False

>>> score = 85
>>> print(score >= 60 and score < 100)
True
```

If Statements

Programs that make decisions — like every app you've ever used

The Basic if Statement

Syntax

```
if condition:
    # code to run
    # if condition is True
```

Two things you must never forget:

1. The colon : at the end of the if line
2. 4 spaces of indentation for the body

Example: video game health check

```
health = 25

if health <= 0:
    print('Game Over!')
    print('Try again...')

if health > 0 and health < 30:
    print('WARNING: low health!')
    print('Find a health pack!')

if health == 100:
    print('Full health!')
```

if / else — Two Paths 🛤️

The else block runs when — and only when — the if condition is False. Exactly one branch always runs.

Syntax

```
if condition:
    # runs if True
else:
    # runs if False
```

Key insight: the condition in an if statement is always a boolean expression — it evaluates to True or False. That's why we learned booleans first!

Example: Netflix access check 🎬

```
is_subscribed = True

if is_subscribed:
    print('Welcome!')
    print('Enjoy Netflix.')
else:
    print('Please subscribe')
    print('to continue.')
```

if / elif / else — Multiple Branches 🌿

Syntax

```
if condition_1:
    # runs first if True
elif condition_2:
    # runs if c1 False, c2 True
elif condition_3:
    # and so on...
else:
    # runs if everything False
```

Only ONE branch ever runs — Python checks each condition top to bottom and stops at the first True.

Example: grade calculator 🎓

```
score = 82

if score >= 90:
    grade = 'A'
elif score >= 80:
    grade = 'B'
elif score >= 70:
    grade = 'C'
elif score >= 60:
    grade = 'D'
else:
    grade = 'F'

print('Grade:', grade)  # B
```

Real World: Spotify Skip Detection 🎵

Behind every Spotify recommendation is a simple if/else chain deciding what counts as a 'real' listen.

Simplified skip detection logic

```
listen_time = 45  # seconds

if listen_time >= 30:
    print('Logging as full listen')
    print('Updating recommendations')
elif listen_time >= 5:
    print('Partial listen logged')
else:
    print('Skipped — not counted')
```

What each branch means:

≥ 30s	Full listen → influences next week's playlist
5-29s	Partial → noted, lighter weighting
< 5s	Skip → tells Spotify you dislike this track

⚡ OPTIONAL CHALLENGE — If Statements

✏️ Pen & Paper

Write a program that checks if a user can create a social media account.

Rules:

- Must be at least 13 years old
- Must agree to terms (True/False)
- If BOTH: 'Account created!'
- If only underage: 'Must be 13+'
- If only terms not accepted: 'Please accept terms'
- If neither: 'Cannot create account'

Write out the if/elif/else structure on paper first.

💻 Try it in IDLE

```
# Build it in IDLE!
age = int(input('Your age? '))
terms = input('Accept terms? (yes/no) ')
terms_accepted = terms == 'yes'

if age >= 13 and terms_accepted:
    print('Account created!')
elif age < 13 and terms_accepted:
    print('Must be 13+')
elif age >= 13 and not terms_accepted:
    print('Please accept terms')
else:
    print('Cannot create account')
```

Work individually • There's no wrong answer — just think it through!

✅ ANSWERS — If Statements

✏️ Pen & Paper

Write a program that checks if a user can create a social media account.

Rules:
- Must be at least 13 years old
- Must agree to terms (True/False)
- If BOTH: 'Account created!'
- If only underage: 'Must be 13+'
- If only terms not accepted: 'Please accept terms'
- If neither: 'Cannot create account'

Write out the if/elif/else structure on paper first.

💻 IDLE Output

```
# Test case 1: age=16, terms='yes'
>>> Account created!

# Test case 2: age=10, terms='yes'
>>> Must be 13+

# Test case 3: age=17, terms='no'
>>> Please accept terms

# Test case 4: age=10, terms='no'
>>> Cannot create account

# Common bug:
# Using if instead of elif
# → Multiple lines print at once!
```

For Loops

Stop repeating yourself — make the computer do it

For Loops with range() 🔢

`for variable in range(n):` → runs the body n times, with variable going from 0 to n-1

range(5) – count up

```
for i in range(5):
    print(i)
# Output:
# 0  1  2  3  4
```

range(1, 6) – custom start

```
for i in range(1, 6):
    print(i)
# Output:
# 1  2  3  4  5
```

range(0, 10, 2) – step by 2

```
for i in range(0, 10, 2):
    print(i)
# Output:
# 0  2  4  6  8
```

range(start, stop, step) — stop is always excluded from the output

More range() Examples 🔢

*Same rule: range(start, stop, step) — **stop is never included**. Predict the output before running!*

range(10, 0, -1) – countdown

```
for i in range(10, 0, -1):
    print(i)

# Counts DOWN:
# 10 9 8 7 6 5 4 3 2 1
# (stops before 0)
# Great for a 🚀 countdown!
```

range(1, 13) – months of year

```
for month in range(1, 13):
    print('Month:', month)

# Month: 1
# Month: 2
# ...
# Month: 12
# (13 is excluded – perfect!)
```

range(2, 20, 3) – step by 3

```
for i in range(2, 20, 3):
    print(i)

# Starts at 2, steps by 3:
# 2  5  8  11  14  17
# (stops before 20)
# Predict: does 20 appear? No!
```

💡 **Negative step: range(start, stop, -1) counts DOWN. start must be greater than stop.**

Looping Over a List 📋

Loop over any list

```
songs = ['Anti-Hero', 'Levitating', 'Espresso']

for song in songs:
    print('Now playing:', song)

# Now playing: Anti-Hero
# Now playing: Levitating
# Now playing: Espresso
```

Loop with position: enumerate()

```
songs = ['Anti-Hero', 'Levitating', 'Espresso']

for i, song in enumerate(songs):
    print(i + 1, '-', song)

# 1 - Anti-Hero
# 2 - Levitating
# 3 - Espresso
```

Accumulator pattern — building a total inside a loop:

```
scores = [85, 90, 72, 96]
total = 0
for s in scores:
    total = total + s
print(total)  # 343
```

Loop Pattern 1 of 3 — Counter

Counter — how many items match a condition?

```
playlist = [3, 5, 7, 2, 8, 1, 6]   # track lengths in minutes
count = 0                          # start counter at 0

for track_len in playlist:         # visit each track
    if track_len > 4:              # check the condition
        count = count + 1          # increment when matched

print(count, 'tracks over 4 min')  # 4 tracks over 4 min
```

1. Set count = 0 BEFORE the loop
2. Loop visits every item in the list
3. if checks condition for each item
4. count + 1 only when condition is True
5. print after the loop for final result

Loop Pattern 2 of 3 — Filter 📂

Filter — collect only the items that meet a condition

```
scores = [72, 88, 55, 91, 63, 80]    # all scores
passing = []                         # start with empty list

for score in scores:                 # visit each score
    if score >= 70:                  # check the condition
        passing.append(score)        # add to results if True

print(passing)                       # [72, 88, 91, 80]
```

INPUT (all scores):

`[72, 88, 55, 91, 63, 80]`

→

OUTPUT (passing only):

`[72, 88, 91, 80]` ← 55 and 63 removed

Loop Pattern 3 of 3 — Build 🏗️

Build — transform every item into a new form

```
names = ['alice', 'bob', 'carol']   # original list
uppercase = []                      # new list to build

for name in names:                  # visit each name
    uppercase.append(name.upper())  # transform and add

print(uppercase)                    # ['ALICE', 'BOB', 'CAROL']
```

Counter — How many?

`count = 0 → count + 1`

Filter — Which ones?

`result = [] → .append if match`

Build — New form?

`result = [] → .append transform`

⚡ OPTIONAL CHALLENGE — For Loops

✏️ Pen & Paper

On paper, trace what this code prints:

```
scores = [85, 42, 91, 67, 73]
total = 0
high = 0
for score in scores:
   total = total + score
   if score > high:
      high = score
print('Total:', total)
print('High score:', high)
```

Then write a loop that prints only the FAILING scores (below 70).

💻 Try it in IDLE

```
# Trace it first, then verify!
scores = [85, 42, 91, 67, 73]
total = 0
high = 0
for score in scores:
    total = total + score
    if score > high:
        high = score
print('Total:', total)
print('High score:', high)

# Print only failing scores:
for score in scores:
    if score < 70:
        print('Failing:', score)
```

Work individually • There's no wrong answer — just think it through!

✅ ANSWERS — For Loops

✏️ Pen & Paper

On paper, trace what this code prints:

```
scores = [85, 42, 91, 67, 73]
total = 0
high = 0
for score in scores:
    total = total + score
    if score > high:
        high = score
print('Total:', total)
print('High score:', high)
```

Then write a loop that prints only the FAILING scores (below 70).

💻 IDLE Output

```
>>> print('Total:', total)
Total: 358
>>> print('High score:', high)
High score: 91

# Failing scores loop:
for score in scores:
    if score < 70:
        print('Failing:', score)

Failing: 42
Failing: 67
```

While Loops

Keep going until the condition changes — not just N times

The while Loop ⏳

Syntax

```
while condition:
    # runs while True
    # update condition!
```

Example: countdown timer

```
count = 5

while count > 0:
    print(count)
    count = count - 1

print('Blast off! 🚀')
# 5  4  3  2  1  Blast off!
```

for loop vs while loop

for loop

Use when you know the number of iterations in advance

while loop

Use when you repeat until a condition changes

⚠️ **Always update the condition variable inside a while loop, or it runs forever!**

Tracing the while Loop — Step by Step 🔍

```
1: count = 5
2: while count > 0:
3:     print(count)
4:     count = count - 1
```

Iteration	count (Line 2)	count > 0? (Line 2)	print(count) Line 3)	Count (Line 4)
1	5	True ✅	5	4
2	4	True ✅	4	3
3	3	True ✅	3	2
4	2	True ✅	2	1
5	1	True ✅	1	0
—	0	False ❌	(loop ends)	—

After the loop exits: print('Blast off! 🚀') runs → Output: 5 4 3 2 1 Blast off!

⚠ The Infinite Loop — Don't Get Stuck!

❌ Infinite loop — runs forever

```
count = 5

while count > 0:
    print(count)
    # oops — forgot to
    # update count!
    # This runs forever.
```

🛑 Press Ctrl+C in IDLE to stop a runaway program.

✅ Fixed — condition eventually False

```
count = 5

while count > 0:
    print(count)
    count = count - 1
    # count updates each time
    # → eventually reaches 0

print('Done!')
```

Rule: something inside the loop MUST change the condition.

Every while loop needs an exit strategy. Ask: 'What will eventually make this condition False?'

Why the Infinite Loop Never Stops 🔄

❌ Broken — count NEVER changes, loop runs forever

Iter	count	count > 0?	print(count)	count updated?
1	5	True ✅	5	❌ No — still 5
2	5	True ✅	5	❌ No — still 5
3	5	True ✅	5	❌ No — still 5
4	5	True ✅	5	❌ No — still 5
...	5	True ✅	5	❌ No — forever

✅ Fixed — count decrements toward 0

Iter	count	print	new count
1	5	5	4
2	4	4	3
3	3	3	2
4	2	2	1
5	1	1	0
—	0	exits	✅ Done

Real World: Wordle-Style Guessing Game 🟩

Wordle keeps looping while guesses remain AND the word hasn't been found — a classic while loop pattern.

```
secret = 'python'
max_guesses = 6
guesses = 0
guessed = False

while guesses < max_guesses and not guessed:
    guess = input('Guess the word: ')
    guesses = guesses + 1

    if guess == secret:
        guessed = True
        print('Correct! You got it in', guesses, 'guesses!')
    else:
        remaining = max_guesses - guesses
        print('Wrong. You have', remaining, 'guesses left.')

if not guessed:
    print('Out of guesses! The word was:', secret)
```

⚡ OPTIONAL CHALLENGE — While Loops

✏️ Pen & Paper

Write a PIN verification loop on paper:

- The correct PIN is 1234
- Allow up to 3 attempts
- After each wrong guess: 'Incorrect. X attempts left.'
- If correct: 'Access granted!'
- If 3 wrong guesses: 'Account locked.'

Trace through your code: what happens when the user enters the correct PIN on attempt 2?

💻 Try it in IDLE

```
# Build the PIN checker!
correct_pin = 1234
max_tries = 3
tries = 0

while tries < max_tries:
    pin = int(input('Enter PIN: '))
    tries = tries + 1

    if pin == correct_pin:
        print('Access granted!')
        break  # we'll learn this next!
    else:
        remaining = max_tries - tries
        if remaining > 0:
            print('Incorrect.', remaining, 'attempts left.')

if tries == max_tries and pin != correct_pin:
    print('Account locked.')
```

Work individually • There's no wrong answer — just think it through!

✅ ANSWERS — While Loops

✏️ Pen & Paper

Write a PIN verification loop on paper:

- The correct PIN is 1234
- Allow up to 3 attempts
- After each wrong guess: 'Incorrect. X attempts left.'
- If correct: 'Access granted!'
- If 3 wrong guesses: 'Account locked.'

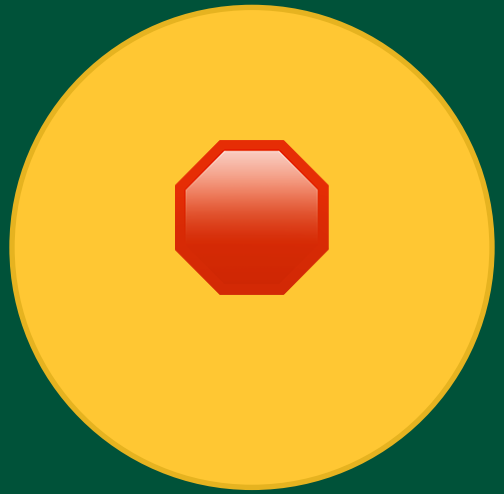
Trace through your code: what happens when the user enters the correct PIN on attempt 2?

💻 IDLE Output

```
# Test: wrong, then correct
>>> Enter PIN: 9999
Incorrect. 2 attempts left.
>>> Enter PIN: 1234
Access granted!

# Test: all 3 wrong
>>> Enter PIN: 9999
Incorrect. 2 attempts left.
>>> Enter PIN: 8888
Incorrect. 1 attempts left.
>>> Enter PIN: 7777
Account locked.

# Test: correct on attempt 1
>>> Enter PIN: 1234
Access granted!
```



Break Statements

Escape the loop the moment you find what you need

The break Statement 🛑

Syntax

```
for item in collection:
    if condition:
        break  # exit immediately
```

What break does:

1. Stops the loop immediately
2. Skips any remaining iterations
3. Jumps to code after the loop

Works in both for and while loops.

Example: find a song in a playlist 🎵

```
playlist = ['Levitating', 'Anti-Hero',
            'Flowers', 'As It Was',
            'Unholy']

target = 'Flowers'

for song in playlist:
    print('Checking:', song)
    if song == target:
        print('Found it! Playing now.')
        break

# Output:
# Checking: Levitating
# Checking: Anti-Hero
# Checking: Flowers
# Found it! Playing now.
```

break in a while Loop — PIN Revisited 🔐

❌ Without break — awkward

```
tries = 0
found = False
while tries < 3 and not found:
    pin = int(input('PIN: '))
    tries += 1
    if pin == 1234:
        found = True
        print('Access granted!')
    else:
        print('Wrong PIN.')
```

✅ With break — clean exit

```
tries = 0
while tries < 3:
    pin = int(input('PIN: '))
    tries += 1
    if pin == 1234:
        print('Access granted!')
        break  # exit immediately
    else:
        remaining = 3 - tries
        print('Wrong.', remaining,
              'tries left.')
```

break removes the need for a separate 'found' flag variable — cleaner and easier to read.

⚡ OPTIONAL CHALLENGE — Break Statements

✏️ Pen & Paper

Write a 'first failing grade finder' on paper:

- List: [88, 92, 67, 75, 45, 81]
- Loop through the scores
- The moment you find a score below 70, print it and stop
- If no failing score is found, print 'All scores passing!'

Bonus: modify it to print the position (index) of the first failing score.

💻 Try it in IDLE

```
grades = [88, 92, 67, 75, 45, 81]

found = False
for i, grade in enumerate(grades):
    if grade < 70:
        print('First fail:', grade)
        print('At position:', i)
        found = True
        break

if not found:
    print('All scores passing!')

# Output:
# First fail: 67
# At position: 2
```

Work individually • There's no wrong answer — just think it through!

✅ ANSWERS — Break Statements

✏️ Pen & Paper

Write a 'first failing grade finder' on paper:

- List: [88, 92, 67, 75, 45, 81]
- Loop through the scores
- The moment you find a score below 70, print it and stop
- If no failing score is found, print 'All scores passing!'

Bonus: modify it to print the position (index) of the first failing score.

💻 IDLE Output

```
>>> for i, grade in enumerate(grades):
...     if grade < 70:
...         print('First fail:', grade)
...         print('At position:', i)
...         found = True
...         break
...

First fail: 67
At position: 2

# Without break (for comparison):
# Would also print: 45 at position 4
# break stops us there
```

Continue Statements

Skip this one — but keep the loop going

The continue Statement ⏭️

Syntax

```
for item in collection:
    if condition:
        continue  # skip to next
    # only runs if not skipped
```

Skip the ads, play the songs 🎵

```
feed = ['song', 'ad', 'song', 'ad', 'song']

for item in feed:
    if item == 'ad':
        continue  # skip ads
    print('Playing:', item)

# Playing: song (x3, no ads)
```

Example: content moderation filter 🛡️

```
comments = [
  'Great video!',
  '',              # empty
  'Love this!',
  '   ',           # whitespace only
  'Thanks!'
]

for comment in comments:
    if comment.strip() == '':
        continue  # skip blank
    print('Posting:', comment)

# Posting: Great video!
# Posting: Love this!
# Posting: Thanks!
```

break vs continue — Side by Side VS

Same list. Same condition. Different behavior.

🛑 break — stop the whole loop

```
nums = [1, 2, 3, 4, 5]

for n in nums:
    if n == 3:
        break
    print(n)

# Output:
# 1
# 2
# (stops — never sees 4, 5)
```

⏭ continue — skip just this one

```
nums = [1, 2, 3, 4, 5]

for n in nums:
    if n == 3:
        continue
    print(n)

# Output:
# 1
# 2
# 4
# 5
# (skips 3, keeps going)
```

⚡ OPTIONAL CHALLENGE — Continue Statements

✏️ Pen & Paper

On paper, predict the output of this code:

```
for i in range(1, 11):
    if i % 2 == 0:
        continue
    print(i)
```

Then rewrite it using break instead — what changes?

Bonus: write a loop that prints all numbers from 1–20, skipping multiples of 3.

💻 Try it in IDLE

```
# Verify your prediction:
for i in range(1, 11):
    if i % 2 == 0:
        continue
    print(i)

# Now with break instead:
for i in range(1, 11):
    if i % 2 == 0:
        break
    print(i)

# Bonus — skip multiples of 3:
for i in range(1, 21):
    if i % 3 == 0:
        continue
    print(i)
```

Work individually • There's no wrong answer — just think it through!

✅ ANSWERS — Continue Statements

✏️ Pen & Paper

On paper, predict the output of this code:

```
for i in range(1, 11):
    if i % 2 == 0:
        continue
    print(i)
```

Then rewrite it using break instead — what changes?

Bonus: write a loop that prints all numbers from 1–20, skipping multiples of 3.

💻 IDLE Output

```
# continue — prints odd numbers:
>>> 1
>>> 3
>>> 5
>>> 7
>>> 9

# break — stops at first even:
>>> 1
# (i=2 triggers break, done)

# Bonus — skip multiples of 3:
# i % 3 == 0 catches 3,6,9...
# Everything else prints:
>>> 1 2 4 5 7 8 10 11 13 14 16 17 19 20
```

🎉 Complete!

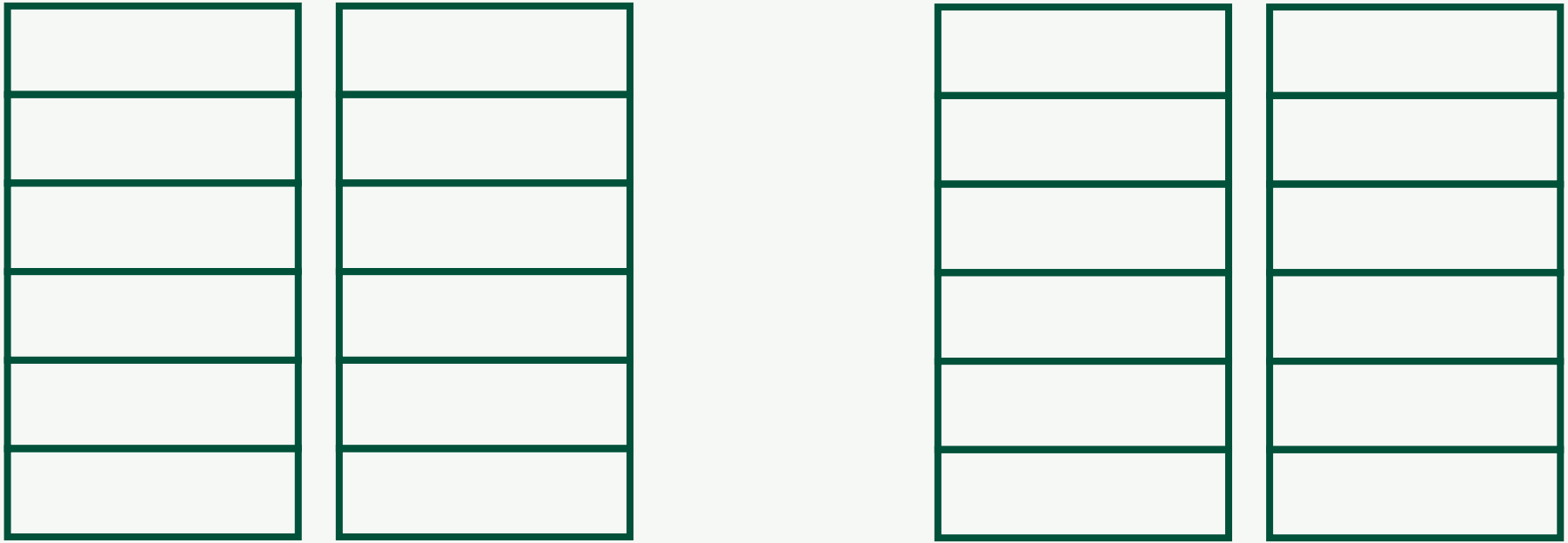
Look what you can do now:

- ✅ Evaluate boolean expressions
- 🔀 Write if / elif / else
- 🔁 Build for loops
- ⏳ Write while loops
- 🛑 Break out of loops
- ⏭️ Continue past items

Next up: Functions → Parameters → Lists → Graphics Capstone 🎨

Questions? Keep working on challenges — or freestyle in IDLE!

Innovation Computer Issues



Front Screen